Airflow Don - A Micromouse Robot

Niklas Peter, *03812287*, Fabian Wallasch, *03725852*, Alessandro Canalicchio, *03796273*, Augustin Ewald, *03754580*

***Abstract*—This report presents the design, implementation, and evaluation of an educational Micromouse robot developed as part of an advanced practical university course. The objective of the project was to realise an autonomous embedded system capable of maze navigation by combining custom hardware, embedded control, and map-based navigation. The robot was built around a custom-designed PCB and a dedicated mechanical chassis, and integrates infrared distance sensors, wheel encoders, motor drive electronics, and wireless debugging functionality. The developed firmware provides low-level hardware control, closed-loop motion control, and autonomous exploration logic. Subsystem-level and system-level tests were carried out to validate the electrical design, sensing, control behaviour, and navigation performance. Within the defined project scope, the robot was able to explore a reduced 6×6 maze, generate an internal map, and use this map for a later goal-directed run.**

***Index Terms*—Embedded robotics, Autonomous navigation, System integration**

I. Introduction

The Micromouse project is based on the classical Micromouse competition, in which a small autonomous robot is required to solve a 16×16 maze. In the present project, however, the task was deliberately reduced to a 6×6 maze with a predefined goal position. This reduced scope was chosen to keep the project feasible within the time and budget of a university engineering project while preserving the core technical challenges of sensing, motion control, localisation, mapping, and autonomous navigation. The central task is therefore not only to move through the maze, but to do so intelligently, reliably, and efficiently within this defined academic scope.

Developing such a robot requires the integration of several engineering disciplines. The robot must be able to sense its environment, estimate relevant motion-related quantities such as position and speed, and execute controlled movements including straight-line driving, wall following, and turning manoeuvres. In addition, communication functions are required for programming, debugging, testing, and visualisation. These capabilities make the Micromouse an appropriate platform for combining embedded programming, electronics, control engineering, and navigation algorithms in a single system.

A key aspect of the project is the use of closed-loop control. Since the robot operates in a constrained maze environment, open-loop motion alone is not sufficient to achieve reliable and repeatable navigation. Sensor feedback must be processed continuously in order to correct deviations in motion and to maintain accurate alignment within the maze. This makes control design a fundamental part of the overall system.

All students were with the Chair of Robotics, Artificial Intelligence and Embedded Systems, Technical University of Munich, Munich, Germany.

In high regards to our leader and mentor, Dr. Alexander Lenz.

From a hardware perspective, the project is centered around a microcontroller-based embedded system. The required functionality includes digital input and output, interrupts, pulse-width modulation, serial communication, quadrature encoder interfaces, and analogue-to-digital conversion. These peripheral features provide the basis for connecting sensors, actuators, and debugging interfaces, and for implementing the software library needed to operate the robot.

Overall, the Micromouse project serves as a practical case study in the development of an autonomous robotic system. It combines the design of hardware and software with the implementation of sensing, control, and mapping strategies. The result is a compact but technically demanding platform that demonstrates how embedded systems and control methods can be applied to autonomous navigation in a structured environment.

A. Aim

The main aim of this project is to design, implement, and evaluate a functioning Micromouse robot that is capable of navigating autonomously through a reduced 6×6 maze environment.

B. Objectives

The objectives of this project are as follows:

- To develop a hardware platform including microcontroller, sensors, actuators, and power supply.
- To implement the required low-level embedded functionality for interfacing with the robot hardware.
- To design and integrate software modules for sensing, motion control, communication, and navigation.
- To implement closed-loop motor and motion control strategies for accurate movement in the maze.
- To test and evaluate the performance of the sensor system, controller, and navigation behaviour.
- To analyse the limitations of the implemented system and identify possible improvements.

II. Conceptual Design and Justification

The conceptual design of the Micromouse was guided by a central systems-engineering question: which overall architecture offers the highest probability of reliable autonomous maze navigation under realistic project constraints. In this context, the challenge was not simply to select individual components, but to define a coherent concept in which sensing, actuation, computation, and software structure support one another. The resulting design therefore prioritised robustness, controllability, and ease of integration over maximum competition speed.

At system level, three criteria were considered particularly important. First, the robot had to be controllable, meaning that commanded motions such as straight driving and turning should be repeatable despite disturbances and asymmetries. Second, it had to be observable, meaning that both the environment and the robot motion should be measurable with sufficient reliability for navigation. Third, it had to be implementable, meaning that the hardware and software complexity should remain compatible with the available time, budget, and manufacturing resources. The final concept was selected as a compromise that satisfies these three requirements simultaneously.

A. Design Requirements and Constraints

From a functional point of view, the robot had to fulfil several basic requirements:

- autonomous movement in a cell-based maze,
- reliable wall detection at the left, front, and right side,
- sufficiently accurate straight driving and turning,
- storage of the explored maze structure,
- execution of a goal-directed run based on the generated map,
- practical support for debugging and commissioning.

In addition to these functional requirements, the project was subject to clear practical constraints. It had to be realised as an academic prototype within limited development time and with a moderate component budget. For this reason, the design objective was not to reproduce a fully competition-optimised Micromouse system, but to realise a complete and technically well-justified autonomous platform. This also affected the problem scope. While the classical Micromouse problem is defined on a 16×16 maze, the implemented system uses a reduced 6×6 maze representation together with a predefined goal configuration. This reduction in problem size was justified because it allowed development effort to be concentrated on the essential system-integration tasks of sensing, control, localisation, and map-based navigation.

Figure 1 shows the physical reduced maze used for the project. Including this reduced maze in the report is useful because it makes the practical task definition more concrete and helps the reader judge the achieved system behaviour against the intended project scope.

B. Locomotion Concept

The selected locomotion concept is a differential-drive robot with two independently actuated drive wheels. This arrangement is particularly appropriate for maze navigation because it combines mechanical simplicity with sufficient manoeuvrability. In contrast to steering-based concepts, no additional steering mechanism is required. In contrast to omnidirectional solutions, the drivetrain remains simple, compact, and easy to model. The robot can therefore execute the fundamental maze manoeuvres directly: straight motion, left and right turns, and turning on the spot.

From a control perspective, the differential-drive concept is advantageous because it reduces the motion problem to the coordinated control of two wheel velocities. However, this benefit only holds if wheel motion is measured. For that reason, wheel encoders were included as an essential part of the concept rather than as an optional refinement. They provide the feedback needed for closed-loop speed regulation, estimation of travelled distance, and monitoring of turn progress [1]. Without this feedback, the robot would depend excessively on open-loop actuation and would accumulate unacceptable motion errors during repeated cell transitions.

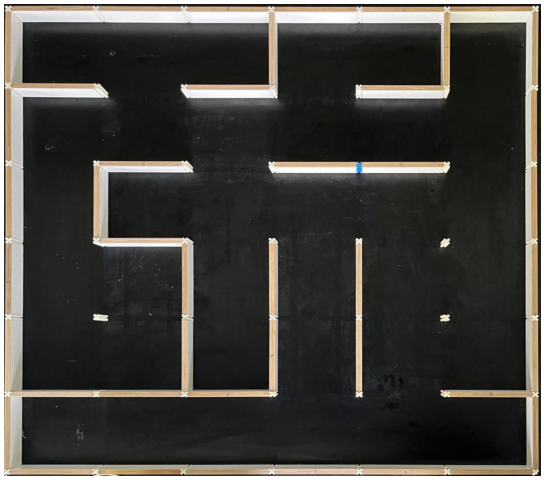

Fig. 1. Reduced maze used for the implemented Micromouse project.

C. Sensing Concept

The sensing concept combines three infrared distance sensors with incremental wheel encoders. The infrared sensors provide environmental information, while the encoders provide motion information. This distinction is important at conceptual level, because successful maze navigation depends both on knowing where walls are and on estimating how the robot itself has moved.

The infrared sensors were arranged to observe the left, front, and right directions. This layout was selected because it captures the minimum external information required for the intended navigation task. The side sensors provide a reference for wall following and lateral correction during straight driving, while the front sensor supports wall detection ahead of the robot and contributes to the interpretation of cell transitions [2]. A larger sensor set could in principle improve redundancy or geometric reconstruction, but it would also increase electrical complexity, software effort, and calibration requirements. For the intended project scope, three sensors represented an effective balance between information content and implementation effort.

The encoders complement the infrared sensors by providing a measure of wheel rotation and therefore of speed and travelled distance. Conceptually, this creates a combined sensing architecture in which external measurements are used to interpret maze structure and internal measurements are used to estimate motion. Such a combination is more robust than relying on only one of the two sensing modalities. Infrared sensing alone would not provide reliable motion estimation,

whereas odometry alone would not detect walls or corridor geometry.

D. Embedded Hardware Concept

The hardware concept is centred around a dedicated microcontroller-based PCB that integrates the major functional subsystems of the robot. This choice was justified by the need for a compact, electrically consistent, and mechanically manageable platform. A single-board design reduces wiring effort, improves reproducibility, and lowers the risk of intermittent faults that commonly arise in loosely wired prototypes. It also supports a cleaner separation between power electronics, sensing, communication, and logic-level processing.

The implemented embedded platform is based on a dsPIC-centered controller board together with a dual H-bridge motor driver, analogue infrared sensors, voltage regulation stages, user-interface elements, and a wireless communication module. At conceptual level, such a microcontroller-based platform is well suited to the application because it combines the peripheral capabilities that are particularly relevant for this task: pulse-width modulation for motor actuation, analogue-to-digital conversion for sensor acquisition, quadrature encoder support for motion feedback, timers and interrupts for real-time scheduling, and serial communication for debugging. By concentrating these functions in one controller, the design remains comparatively compact and avoids unnecessary interface overhead [3].

The power architecture follows the same logic of functional integration. Since the drive system and the logic-level electronics have different electrical requirements, voltage regulation is necessary to provide stable operating conditions for sensors, communication hardware, and control electronics. In addition, wireless UART communication was included as part of the concept because a mobile robot benefits strongly from untethered debugging during experiments in the maze. Although this feature is not strictly required for autonomous motion, it significantly improves observability during development and therefore supports the practical implementation process [4].

E. Software and Navigation Concept

The software concept follows a layered architecture in which hardware access, motion control, and navigation logic are separated into distinct levels of abstraction. This structure was chosen because the project combines tasks of very different character. At lower level, the software must handle peripheral configuration, sensor acquisition, and actuator output. On top of this, the control functions regulate wheel motion and provide reusable movement primitives, while the navigation layer represents the maze, decides where to move next, and manages the transition between exploration and goal-directed motion. A layered design reduces coupling between these concerns and thereby improves maintainability and testability.

At navigation level, a map-based approach was selected instead of a purely reactive one. This decision is conceptually important. A reactive robot can avoid immediate obstacles, but it does not necessarily solve a maze efficiently or even correctly when revisiting already explored regions. By contrast, a map-based approach allows the robot to store cell connectivity, distinguish between local sensor observations and global maze orientation, and compute paths through already known territory. This is much more suitable for a Micromouse-type task, where successful behaviour depends not only on instantaneous wall detection but also on memory and route selection.

The chosen strategy therefore consists of two logically separate phases. In the first phase, the robot explores the accessible maze and constructs an internal representation of the discovered cell connections. In the second phase, this stored representation is used to drive toward a predefined goal. This division is justified because it separates the difficult task of acquiring environmental knowledge from the later task of exploiting that knowledge. It also reflects a clear experimental structure: the first run evaluates sensing, localisation, and map consistency, whereas the second run evaluates the usefulness of the generated map for purposeful navigation.

F. Justification of the Overall Approach

Taken as a whole, the selected concept is justified not because each individual element is maximally sophisticated, but because the elements fit together in a technically coherent way. The differential-drive arrangement provides a simple and controllable mechanical basis. The encoder and infrared sensor combination provides the minimum feedback necessary for closed-loop motion and wall-based interpretation of the maze. The microcontroller-based PCB supplies the required embedded functionality in a compact implementation. The layered software structure and map-based navigation strategy provide an appropriate computational framework for autonomous maze traversal.

This makes the overall design well suited to the aims of the project. Rather than pursuing a narrowly optimised high-speed competition robot, the concept emphasises the successful integration of electronics, embedded software, control, and navigation into a complete autonomous system. Such a design is particularly appropriate in an academic context, because it exposes the main engineering trade-offs clearly while remaining feasible to implement, test, and evaluate in a rigorous manner. The following chapters build on this conceptual basis by describing the detailed hardware and software realisation and the implemented controller design.

III. Hardware Design

The Micromouse hardware was developed as a compact embedded system that integrates power supply, sensing, actuation, computation, communication, and user interaction on a single custom PCB. The purpose of this hardware platform is not merely to connect the required components, but to provide a reliable physical basis for closed-loop motion control and autonomous maze navigation. Since the robot operates as a mobile real-time system, the hardware design had to satisfy several requirements simultaneously: sufficient actuation power for the drivetrain, stable low-voltage supplies for the logic and sensor subsystems, low-latency access to sensor and

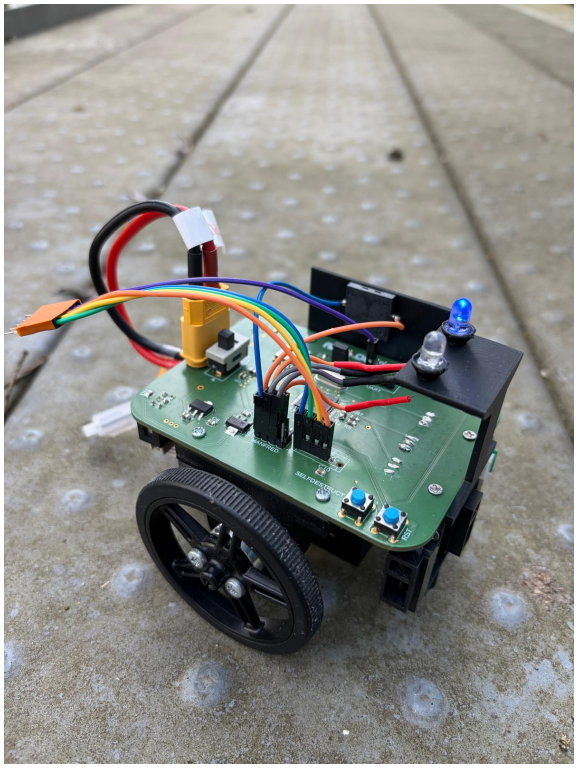

Fig. 2. Final assembled version of the Micromouse robot.

encoder signals, and practical interfaces for programming and debugging.

At a high level, the implemented hardware can be divided into five functional blocks: the controller and timing hardware, the power-supply subsystem, the motor-drive and encoder subsystem, the infrared sensing subsystem, and the communication and user-interface subsystem. These blocks are electrically distinct, but they were designed to operate as one coordinated system. In particular, the PCB serves as the central integration platform between the battery input, the motors, the sensors, the microcontroller, and the external interfaces. This approach is advantageous in a mobile robot because it improves reproducibility, reduces wiring effort, and lowers the likelihood of intermittent connection faults that are common in loosely assembled prototypes.

Figure 2 shows the final assembled Micromouse. The image illustrates the compact overall construction of the robot and the close integration of the mechanical platform with the electronics.

A. Controller and Timing Hardware

The central processing element of the robot is a `dsPIC33FJ128MC804` [3]. This controller was selected because it combines the peripheral functions required by the application in a single device. For the Micromouse, these functions are particularly important: PWM generation is needed for motor actuation, analogue inputs are required for the infrared sensors, quadrature encoder interfaces are needed for odometry, timers and interrupts are required for deterministic periodic execution, and UART communication is needed for debugging and monitoring. Using one controller that already supports these functions natively simplifies the circuit design and reduces the number of additional interface components.

The controller is driven by an external 16 MHz crystal together with the associated load capacitors. This clock source provides a stable timing reference for the microcontroller and therefore supports reliable sampling, control-loop timing, and UART communication. In the firmware, the oscillator is combined with the internal PLL in order to obtain a sufficiently high operating frequency for real-time processing [3]. This clocking concept is well suited to the robot, because the control loop, sensor handling, and navigation logic all depend on predictable timing behaviour.

B. Power Supply Architecture

The robot is powered from a 7.4 V battery, which forms the primary energy source of the system. From this input, the power is distributed into separate voltage domains according to the electrical requirements of the connected components. This separation is necessary because the drive system, the analogue sensors, and the digital electronics do not operate at the same supply voltage and do not have the same sensitivity to electrical disturbances.

The unregulated 7.4 V battery voltage is used as the motor-supply domain, so that the H-bridge can provide sufficient drive voltage to the two DC motors. In operation, the effective motor drive level is limited by PWM so that the applied actuation remains compatible with the motor rating of 6 V [1]. A first regulator, implemented with an `LM7805`, generates a 5 V rail [5]. This 5 V domain supplies the three infrared distance sensors and thereby provides the operating voltage for the analogue wall-detection subsystem [2]. From this 5 V rail, a second regulator, implemented with an `LD1117`, generates the 3.3 V logic supply [6]. The 3.3 V domain is used by the `dsPIC33FJ128MC804`, the `RN4871` Bluetooth module, and the remaining low-voltage digital electronics [3], [4].

This staged regulation concept offers two main advantages. First, it ensures that each subsystem is operated within its intended electrical range. Second, it reduces the direct coupling of motor-related disturbances into the analogue and digital subsystems. The supporting passive components around the power stage therefore play an important role. Larger capacitors at the regulator inputs and outputs help stabilise the supply rails during load changes, while smaller decoupling capacitors placed close to integrated circuits suppress local high-frequency disturbances. Together, these measures improve the robustness of the electrical supply under the dynamic conditions of robot operation.

C. Drive Electronics and Encoder Interface

The mechanical locomotion concept of the robot is based on two independently driven DC motors in a differential-drive arrangement. Electrically, both motors are actuated through a `TB6612FNG` dual H-bridge [7]. This component is appropriate for the project because it combines two bidirectional motor channels in one compact device and supports PWM-based control with separate direction inputs. In the implemented

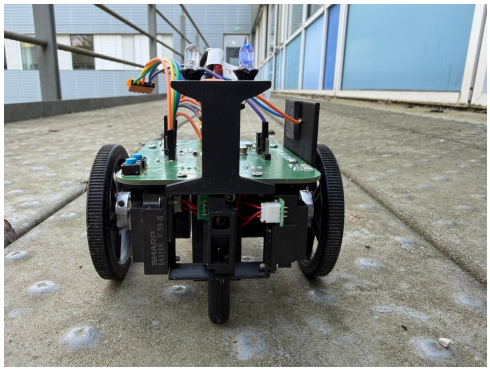

Fig. 3. Front view of the Micromouse showing the forward sensor arrangement.

system, each motor is driven by one PWM signal and two logic-level direction signals, which makes it possible to realise forward motion, reverse motion, coasting, and active braking.

The motor driver is not only important for producing motion, but also for enabling controllable motion. Since the Micromouse must drive straight, execute turns, and stop repeatably, the electrical motor interface has to support both continuous duty-cycle control and rapid mode changes. The chosen H-bridge meets this requirement and integrates naturally with the PWM hardware of the dsPIC [3], [7].

Each motor is equipped with a quadrature encoder, and both encoders are connected directly to the dedicated QEI peripherals of the microcontroller. This is a particularly important design decision, because the navigation strategy depends on accurate motion feedback. The encoder signals provide the information required for wheel-speed estimation, travelled-distance measurement, and turn-progress detection [1]. By connecting the encoders to hardware-supported quadrature interfaces, the design reduces software overhead and improves the reliability of odometry acquisition.

D. Infrared Sensing Hardware

The environmental sensing system consists of three analogue infrared distance sensors positioned to observe the left, front, and right directions. This arrangement was selected because it provides the minimum external information required by the implemented control and navigation strategy. The two side sensors are used for wall-based trajectory correction during straight motion, while the front sensor is used to detect walls ahead and to support the interpretation of cell transitions inside the maze [2].

Figure 3 shows the front view of the robot. This image is particularly useful because it makes the practical placement of the forward-facing sensor arrangement visible and illustrates the geometric relation between the sensor system and the maze walls encountered during operation.

Electrically, the three sensors are connected to analogue inputs of the dsPIC. In the implemented pin assignment, the right, middle, and left sensors are connected to channels AN0, AN1, and AN8 respectively. The ADC is operated in 12-bit mode and scans all three channels continuously. A DMA channel transfers the conversion results directly into memory, so that recent sensor values are available without requiring the processor to handle every sample explicitly [3]. This arrangement is beneficial in a real-time embedded system because it combines regular sensor acquisition with low processor overhead.

The supporting analogue circuitry around the sensor inputs is comparatively simple, but still important. Since infrared wall sensing is used continuously during operation, stable sensor supply voltage and clean signal acquisition are essential. The chosen 5 V sensor supply and the local supply-stabilisation components therefore contribute directly to measurement reliability [2].

E. Communication and User Interface Hardware

In addition to the motion and sensing electronics, the robot includes dedicated interfaces for programming, debugging, and local user interaction. An ICSP programming header is provided for flashing the microcontroller during development. This interface is essential for a custom embedded board, because it gives direct access to the controller independently of the higher-level communication interfaces.

For runtime communication, the robot uses a UART interface together with an `RN4871` Bluetooth Low Energy module [4]. This module was selected as a practical wireless bridge for serial debug output. During development, this interface made it possible to observe internal variables such as sensor readings, controller states, and other diagnostic values without physically tethering the robot. For a maze-running system, this is particularly convenient because a wired connection would restrict the motion of the platform and complicate testing.

The user-interface hardware consists of push buttons and several status LEDs. The buttons provide local control functions such as start, reset, and operation changes, while the LEDs provide direct visual feedback during commissioning and testing. In practice, these simple components proved highly useful because they allowed rapid confirmation of system states without requiring a Bluetooth connection or oscilloscope measurement. The LED circuits are supported by current-limiting resistors, and the button-related logic is stabilised by suitable biasing and pull-up arrangements so that defined logic levels are maintained during operation.

F. PCB Realisation and Supporting Components

The complete electronics were implemented on a dedicated PCB rather than on separate modules. This decision is important from an engineering perspective, because the PCB is not merely a mechanical carrier but the central means of electrical integration. It defines how the power input, regulators, controller, motor driver, sensors, communication interfaces, and auxiliary components interact as one system. A custom PCB also improves repeatability and serviceability, since all major electrical connections are fixed and clearly structured.

Figure 4 shows the assembled PCB of the Micromouse electronics. The image illustrates the practical realisation of the integrated hardware concept and shows how the major components are concentrated on one board.

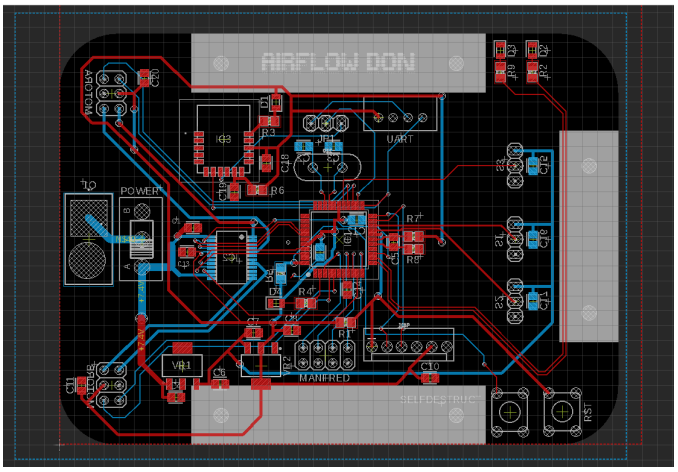

Fig. 4. Assembled PCB of the Micromouse electronics.

Besides the major active components, a considerable part of the board functionality depends on standard supporting parts. Ceramic decoupling capacitors, regulator support capacitors, crystal load capacitors, current-limiting resistors, and biasing resistors do not define the robot behaviour by themselves, but they are essential for stable operation. In particular, they ensure clean supply rails, reliable clock generation, protected LED operation, and defined logic states at sensitive inputs. For this reason, the quality of the final hardware design depends not only on the selection of the main integrated circuits, but also on the correct use of these supporting electronic components.

IV. Software Design

The firmware was developed as a real-time embedded system that links low-level peripheral handling, closed-loop motion control, and map-based navigation. Its purpose is not only to command the motors and read the sensors, but also to maintain a consistent internal representation of the robot state while decisions are made during motion. In the Micromouse application, this is particularly important because sensing, control, and navigation cannot be treated as independent tasks: the quality of the maze map depends on accurate motion execution, while the correctness of the motion commands depends on the current navigation state.

A. Software Architecture

The implemented software follows a layered structure with three functional levels. At the lowest level, a hardware-oriented layer configures and accesses the dsPIC peripherals, including GPIO, PWM, timers, ADC, DMA, UART, and the quadrature encoder interfaces. On top of this, a driver and control layer provides reusable functions for motor actuation, encoder-based speed measurement, wall detection, Bluetooth communication, and closed-loop drive control. The highest level is the navigation layer, which stores the internal maze representation, estimates cell transitions, manages the exploration frontier, and selects the next motion command.

This separation was chosen in order to reduce coupling between tasks of different character. Hardware-specific details such as ADC configuration or PWM timing are isolated from the navigation logic, while the navigation layer operates in terms of cells, headings, and movement primitives rather than raw peripheral registers. Such a structure improves clarity and makes it easier to test and modify individual subsystems during development.

B. Execution Model and Timing

After power-up, the firmware performs a defined initialization sequence in which the clock, I/O pins, communication interfaces, timers, PWM hardware, motor and encoder interfaces, ADC with DMA support, and the controller state are configured before exploration begins [3]. After this initialization, the main loop remains empty and the system operates primarily in an interrupt-driven manner.

The central periodic task is executed every $10\,\mathrm{ms}$ by a timer interrupt. In this interrupt, the controller update is executed and the exploration logic checks whether the robot has reached the center of the next cell. This design was selected because both control and navigation depend on a deterministic update interval. Continuous infrared sampling is handled separately by the ADC and DMA hardware, so that current sensor values are always available in memory without requiring the processor to manage every conversion directly. In addition, encoder feedback is acquired via dedicated hardware peripherals, which reduces processor load and improves the reliability of motion estimation. A second timer interrupt is used for auxiliary functions such as audio signalling and visual feedback, but it is not part of the core navigation loop.

C. State Representation and Maze Model

The navigation layer maintains a central software state that contains the current cell position, the current global heading, the distance travelled since the last detected cell centre, and the exploration status of the maze. The maze itself is represented as a 6×6 cell grid. For each cell, the software stores which neighboring cells are reachable and whether the cell has already been explored. This representation is intentionally simple, but it is sufficient for the reduced project scope because the maze can be treated as a graph of connected cells.

In addition to the maze map, the software stores temporary path information and a frontier data structure for cells that still need to be visited. This frontier is implemented in a queue-like container but operated in a Last-In-First-Out manner, so that it behaves like a stack in practice. This gives the exploration behaviour a depth-first character, while still allowing the robot to compute a route through already known cells when the next target is no longer directly adjacent.

D. Detection of Cell Centers

A reliable estimate of the cell centre is essential for maintaining consistency between the physical robot position and the internal maze coordinates. If a new navigation decision is triggered too early or too late, the stored map and the real robot location diverge. For this reason, cell-centre detection is treated as a dedicated software function rather than as an implicit consequence of motor timing.

The implementation uses two complementary criteria. First, the travelled distance is estimated continuously from the encoder information, and a new cell is assumed once approximately one cell length has been covered. Second, the front infrared sensor is monitored during straight driving. A sufficiently strong front-wall reading can also trigger the next exploration step, since the wall geometry provides an additional indication that the robot has reached the relevant point within the cell. In this way, the software combines odometry-based estimation with environmental sensing in order to reduce the risk of losing orientation.

E. Exploration Strategy and Direction Handling

At the beginning of a run, the software initializes the robot with a start position, a start heading, and an empty maze map. The start cell is marked as explored, and the first candidate target cells are inserted into the frontier structure. Whenever the robot reaches a new cell centre, the software evaluates the left, right, and front wall sensors relative to the current robot heading. Each detected open direction is then converted from the local frame of reference into the corresponding global maze direction and stored as a valid neighbor connection in the map.

This distinction between local and global orientation is central to the software design. Locally, the robot reasons in terms of front, left, right, and rear because this matches the sensor arrangement and the available turning commands. Globally, however, the maze must be stored consistently in terms of north, east, south, and west. The navigation logic therefore includes an explicit transformation between both representations. This ensures that sensor observations made from different robot headings can still be integrated into one coherent map.

F. Path Selection and Goal-Directed Operation

After updating the map of the current cell, the software determines the next target. If the next frontier entry is directly adjacent, the required action can be derived immediately as a local movement decision. If the target lies elsewhere in the already explored part of the maze, the software computes a path through the known map by recursively searching through the reachable neighboring cells and selecting the shortest currently known route. The resulting sequence of global directions is then executed step by step by combining turning primitives with straight motion.

The overall navigation behaviour is divided into two phases. In the first phase, the robot explores the accessible maze and constructs the map incrementally. Once no unexplored frontier cells remain, exploration is marked as complete and the robot uses the stored map to return to the start region. In the second phase, triggered separately by the user, the robot calculates a path from its current position to the predefined goal and follows this route through the known maze. This two-phase structure was chosen because it separates map generation from map exploitation and thereby makes the system behaviour easier to test and interpret.

V. Controller Design

Reliable motion control is a central requirement for autonomous maze navigation, since the robot must move accurately and repeatably in a constrained environment. In practice, open-loop motor actuation is not sufficient, because wheel slip, drive asymmetries, and other disturbances can cause deviations from the intended path. For this reason, the controller was implemented as a cascaded architecture consisting of an inner velocity-control loop and an outer correction layer for maintaining straight motion. The inner layer regulates the wheel velocities based on encoder feedback, while the outer layer applies an additional trim-based correction to improve the trajectory during straight driving. In addition to straight-line motion, the controller design also includes dedicated turning functions required for maze navigation.

A. PI Control for Drive Speed

The purpose of the inner control loop is to regulate the wheel velocities during straight driving and thereby provide accurate and repeatable motion. Instead of commanding the motors in open loop, the system controls the wheel speeds explicitly, which improves robustness against disturbances and variations in motor behaviour. Since the robot is driven by two separate motors that are not perfectly identical, separate speed controllers for the left and right wheel are required [8]. This is not merely convenient, but necessary for balanced forward motion.

The measured variable of each controller is the linear wheel velocity, which is derived from the corresponding integrated quadrature encoder [1]. For each wheel, the encoder count is accumulated continuously, and the count increment over one sampling interval is used to estimate the rotational motion of the wheel. Based on the known number of encoder counts per wheel revolution and the wheel circumference, the linear velocity can be calculated as

$$v = \frac{\Delta N}{N_{\mathrm{rev}}} \cdot \frac{\pi d}{T_s}, \tag{1}$$

where ΔN denotes the encoder count difference during one sampling interval, N_{rev} is the number of counts per wheel revolution, d is the wheel diameter, and T_s is the sampling time. This estimated wheel velocity serves as the feedback signal for the controller.

The wheel velocity is compared with a desired target velocity, and the resulting control error is processed by a proportional-integral controller. The proportional part provides an immediate response to the current speed error, while the integral part compensates persistent deviations over time and reduces steady-state error [9]. In the implemented controller, the integral contribution is additionally bounded in order to limit windup effects and prevent excessive output accumulation.

For the final implementation, the wheel-speed controller was operated with the empirical parameter set $K_P = 1.4$ and $K_I = 16.0$ at a sampling time of $T_s = 0.01\,\mathrm{s}$. These values were not derived from a formal plant model, but were selected experimentally during commissioning as the best compromise

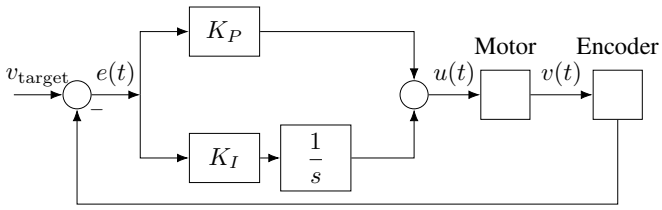

Fig. 5. Block diagram of the inner PI wheel-speed controller.

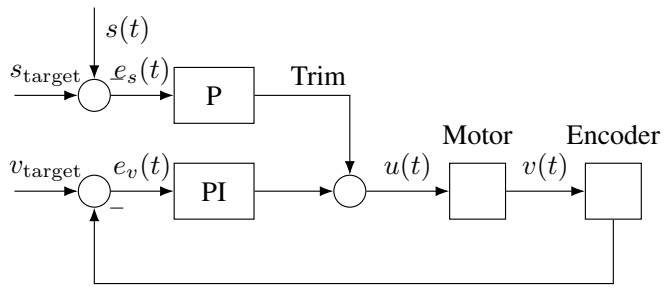

Fig. 6. Generalized cascaded controller structure with inner PI speed control and outer trim correction.

between visible speed tracking, stable response, and limited stuttering. A summary of the gain adjustment process is provided in Appendix A.

Figure 5 illustrates the standard feedback structure of the implemented wheel-speed controller for one wheel. The same principle is applied independently to both motors.

This inner wheel-speed regulation forms the basis of the overall cascaded drive controller and provides the controlled input on top of which the outer trajectory-correction layer operates.

B. Trajectory Correction by Cascaded Trim Control

While wheel-speed regulation ensures that both motors track their velocity references, it does not by itself guarantee reliable straight motion in the maze. In addition to stable wheel velocities, the robot must maintain a consistent trajectory within the corridor and ideally follow the middle line between the surrounding walls. This additional requirement motivated the implementation of an outer correction layer.

In practice, several effects were observed that caused the robot to deviate from its intended path. These included unequal motor behaviour, wheel slip on the maze surface, and the accumulation of small trajectory errors during straight motion. Without additional correction, such deviations would eventually cause the robot to drift into a wall, which would compromise the execution of the overall maze-solving task. Since reliable straight motion is a prerequisite for any higher-level navigation strategy, an additional correction mechanism was required.

To address this, the drive controller was extended into a cascaded structure. The inner PI loop regulates the wheel velocities, while an outer controller modifies the wheel commands in order to stabilize the robot along a straight trajectory. The interaction of the inner velocity loop and the outer trim controller is shown in Fig. 6. The outer layer uses the left and right side infrared sensor readings as correction inputs [2]. These signals are raw 12-bit sensor values rather than metric distance measurements. In theory, the values lie in the range from 0 to 4095, although in practice they reach only up to approximately 2300 when the sensor is as close to the wall as possible.

C. Signal Conditioning and Slew-Limited Actuation

In addition to the cascaded controller structure described above, several practical signal-conditioning measures were introduced in the implementation in order to improve robustness and smoothness on the physical robot. While the inner PI controller and the outer trim-based correction define the main control logic, their direct application on real hardware can still lead to undesirable behaviour if noisy feedback signals or abrupt command changes are not handled appropriately [10]. For this reason, the controller was extended by filtering and output-shaping mechanisms that improve the stability of the overall drive behaviour.

A first measure concerns the encoder-based wheel-speed feedback used by the inner PI loop. Although the wheel velocity is computed from encoder counts over a fixed sampling interval, the resulting estimate may still fluctuate due to quantisation effects and small variations between successive samples. To reduce the influence of such fluctuations, the measured speed is passed through a low-pass filter before it is used for control. This produces a smoother feedback signal and prevents the PI controller from reacting too aggressively to short-term variations in the estimated wheel velocity [9].

A similar approach is used for the trim-based correction generated by the outer control layer. Since the trim signal is derived from side sensor readings, it may change abruptly when the measured wall position varies from one control step to the next. Applying this correction directly to the wheel command would result in sudden lateral adjustments and unnecessarily nervous motion. Therefore, the trim signal is also low-pass filtered before it is added to the inner controller output. This leads to a smoother trajectory correction and improves the stability of straight driving within the maze corridors.

After the proportional-integral action and the trim correction have been combined, the resulting motor command is further constrained to the admissible actuator range. In the implementation, the normalized command is limited to the interval from -1 to 1, which reflects the valid operating range of the motor interface. This saturation prevents invalid or excessively large actuation requests and ensures that the controller output remains compatible with the physical limitations of the drive system.

Finally, the bounded controller output is not applied to the motors in an instantaneous step. Instead, the command is updated gradually by means of slew-rate limiting, such that only a limited change is allowed in each control step. This reduces abrupt transitions in motor actuation and leads to smoother acceleration and deceleration behaviour. In the context of the Micromouse robot, this is particularly useful because rapid command jumps can amplify mechanical vibrations and reduce

the stability of straight driving.

Overall, these additional implementation measures do not introduce a new control layer, but they significantly improve the practical behaviour of the existing cascaded controller. By filtering noisy intermediate signals, constraining the actuator command, and limiting rapid changes in the applied motor input, the controller becomes more robust and better suited for reliable operation on the real robot.

D. Additional Motion Functions

In order to control turns, when a wall is encountered the `turnLeft90()`, `turnRight90()`, or `turn180()` function is called. These three functions operate similarly: they reset the wheel-speed controller to prevent residual control action from straight driving from affecting the turn, store the current encoder reading for each motor, and set the controller into the corresponding turn mode. Whenever the controller is in turn mode and the `updateController()` function is called by the timer interrupt, the turn controller is updated. It computes the absolute encoder-count difference between the current and initial values for each wheel and uses the average of both wheels as the measure of turn progress. If the target encoder-reading difference required for a $90°$ or $180°$ turn has not yet been reached, the motors are commanded to rotate in opposing directions with a predefined constant speed in open-loop actuation. If the target encoder reading has been reached and exploration is not yet finished, the internal state distance used by the exploration module is reset and the controller is returned to drive mode. Once the target encoder difference is reached, the motors are briefly stopped and the wheel-speed controllers are reset again to ensure a clean transition. A short forward motor command is then applied to reduce mechanical oscillations and stabilise the robot before resuming motion. If exploration has already been completed, the robot remains stopped.

VI. Tests

The tests carried out during development began with general electrical verification of the assembled PCB in order to ensure correct soldering and reliable basic operation. At an early stage, a multimeter was used to check the main power path across the board, starting from the 7.4 V battery, through the H-bridge and the first voltage regulator, which generated the 5 V supply. This 5 V line was then verified as the supply for the three analog infrared sensors and as the input to a second regulator providing 3.3 V for the remaining digital components, including the microcontroller. After these static supply checks, the verification was extended by oscilloscope measurements in order to assess dynamic electrical signals at relevant module interfaces, for example at PWM outputs and other time-varying signals. In this way, the initial test phase did not only confirm the presence of the required supply voltages, but also provided a general board-level check that the major components received the expected electrical signals before more specific tests were carried out.

The purpose of this chapter is to describe how the individual subsystems were validated during development. For this reason, the emphasis is on the test procedures, debugging steps, and validation scenarios used during commissioning. The final technical outcomes and the selected operating parameters are summarized separately in the following results chapter.

A. Sensor Tests

The sensor tests were comparatively simple, since the infrared sensors functioned reliably from the beginning and therefore did not require extensive dedicated tuning [2]. After the sensors had been connected to the system, all three raw sensor signals were first observed via UART output in order to verify that valid readings were received and updated correctly during operation. The initial goal of these tests was therefore not detailed calibration, but confirmation that the sensing chain from analogue input to software interpretation worked as intended.

To evaluate the sensors under realistic operating conditions, the robot was then placed inside the maze and positioned close to surrounding walls. These measurements were used to compare free-space behaviour with the more relevant operating range inside the maze and to determine whether the sensors provided a sufficiently clear distinction between wall and no-wall conditions for the intended navigation task. Since the software relies primarily on threshold-based wall detection rather than on precise metric distance reconstruction, this practical distinction was more important than an exact calibration curve.

For easier debugging during deployment, the on-board LEDs were also used as direct visual indicators of wall detection, so that the sensor behaviour could be checked without relying on a UART connection. In this configuration, the blue LED indicated detection by the left sensor, the red LED indicated detection by the right sensor, and the green LED indicated detection by the front sensor. This made it possible to verify the switching behaviour of the wall-detection logic directly on the robot during operation in the maze.

Overall, the sensor system required only limited dedicated testing, since all three sensors worked essentially out of the box. The main outcome of the sensor tests was therefore not a redesign of the sensing concept, but the validation of the existing sensor arrangement and the derivation of a practical detection threshold for later use in the control and navigation logic.

B. Controller Tests

The controller tests were carried out incrementally, beginning with the inner wheel-speed control and later extending to the wall-based trim correction used during straight driving. As a prerequisite, the motor actuation had already been implemented in normalized form, such that motor commands could be specified in the range from -1 to 1 and were automatically limited to the admissible actuation range. Building on this interface, the first controller-related tests focused on verifying that a commanded translational speed in $\mathrm{mm/s}$, provided through the function `setDriveSpeedMmps()`, could be mapped to a corresponding fixed motor actuation and that the drivetrain responded accordingly. At this stage, the purpose was not yet closed-loop regulation, but rather to confirm that

the command path from software input to physical wheel motion worked as intended.

For observation during testing, a verbose UART output was available from the beginning and was transmitted every $10\,\mathrm{ms}$ from the timer-interrupt-based controller update. This output included the measured left and right wheel velocities, the sensor readings, and further internal variables relevant for debugging, so that no additional instrumentation had to be added during tuning. Once the basic control path had been verified, the structure was extended to a closed-loop PI controller for wheel-speed regulation inside `updateWheelSpeedController()`, which was called periodically from `updateController()`. The tuning process then began with both controller gains set initially to zero. First, a proportional sweep was carried out on the right wheel with $K_I = 0$ and a simple two-step speed command, in order to identify the range in which visible speed regulation began and the point at which stuttering became excessive. After a suitable proportional range had been identified, the integral gain was increased incrementally while a wider cyclic speed sequence was applied. This made it possible to compare the reduction of steady-state deviation against the onset of increasingly aggressive behaviour. The left wheel was then tuned starting from the parameter region identified on the right side in order to check whether a common parameter set could be used. A summary of this empirical tuning sequence and the associated qualitative observations is provided in Appendix A.

In principle, separate parameter sets for both wheels would be justified, since the two motors and the associated drivetrain are not perfectly identical. For this reason, both wheels were initially considered independently during tuning. The observations summarized in Appendix A show, however, that both sides behaved similarly enough around the selected tuning region that a common parameter set could be used in the final implementation. This simplified the controller structure while still allowing satisfactory straight driving within the scope of the project.

After the single-wheel tests, both controlled wheels were evaluated together on different surfaces, including the floor, the maze surface, and a desk. The purpose of these tests was to determine whether the robot still exhibited sufficiently straight motion under changing contact conditions. Since the drive behaviour remained acceptable on all tested surfaces, this part did not require extensive further investigation.

Once the inner PI controller had reached a stable and usable state, testing was extended to the wall-based trim correction. A first proof-of-concept test was carried out in open space, where no surrounding wall was present and both side sensors therefore produced approximately zero readings. Under these conditions, no trim correction should be generated. To introduce a controlled lateral disturbance, a wooden slab was manually placed near either the left or the right side sensor while the robot was driving. The expected behaviour was that the robot would steer away from the detected obstacle. During this test, it was observed that the correction direction was initially inverted, such that the robot turned toward the slab instead. This revealed a sign error in the correction logic, which was then reversed.

After this initial validation, the trim-based correction was tested under more realistic maze conditions. For this purpose, the robot was placed in the maze and repeatedly driven along a hardcoded square path. The test logic was implemented such that the robot drove straight until a front wall was detected, then stopped the straight-drive controller, executed an approximately 90° left turn using `turnLeft90()`, and continued driving. This setup proved useful because the turning motion was only approximate, so that the robot did not always enter the next corridor in perfect alignment. As a result, the subsequent straight segment required active correction in order to return the robot toward a stable path within the corridor. The square path therefore provided a repeatable test scenario for evaluating whether the correction logic could compensate such misalignment robustly.

Initially, the trim correction inside `updateWheelSpeedController()` had been implemented as a PI controller. However, no satisfactory parameter set could be found that produced both fast and stable correction behaviour. In practice, the robot exhibited pronounced lateral oscillations and followed a snake-like path along the corridor. After reevaluation in discussion with Dr. Alexander Lenz, the trim controller was reduced to a proportional controller. This modification led to significantly improved behaviour and allowed the correction to react more directly to lateral deviations without introducing the previously observed oscillatory motion.

In addition, the correction logic had to account for the fact that not all maze sections provide the same lateral wall structure, so trim-based corrections were applied only when a valid side-wall reference was available. To improve practical robustness, filtering and slew-rate limitation were also introduced during tuning in order to reduce noise sensitivity and abrupt actuator changes.

Overall, controller testing was considered successful once the wheel-speed controller tracked commanded velocities reliably, the robot maintained sufficiently straight motion in practice, and the wall-based outer correction enabled repeated square-path execution in the maze without substantial loss of alignment, in both single and double wall regimes. These tests established a sufficiently stable basis for later sensor-related testing and further system integration.

C. Navigation Tests

The navigation behaviour was validated experimentally in several incremental stages. Since no dedicated automated test framework was implemented in the code, verification was mainly carried out through practical trials with the physical robot. At the beginning, only a very small maze and simple controller settings were used in order to reduce complexity and make individual effects easier to observe. This made it possible to focus first on the relation between sensor feedback, travelled distance, and the internal cell-based position estimate.

An important early step was to drive the robot at very low speed and stop it whenever the software detected that a cell centre had been reached. At the same time, UART output was used to indicate when such a detection occurred. This allowed

a direct comparison between the internal software decision and the real physical position of the robot in the maze.

In addition to odometry-based checks, the front sensor values were examined while the robot was positioned at cell centres. Based on these measurements, a suitable value range was defined in which the sensor feedback could be interpreted as a valid indication of a cell centre. This helped to make the detection more robust and reduced the risk of orientation errors during exploration. After these basic mechanisms worked reliably, the overall difficulty of the test scenarios was increased step by step.

The exploration logic was also introduced incrementally. At first, the robot was only able to explore direct neighboring cells, without any recursive path calculation through the known maze. This simplified the behaviour and made it easier to isolate problems in localisation, queue handling, and wall detection. Only after these subsystems appeared sufficiently robust, the recursive search was implemented, allowing the robot to calculate a path to queued cells that were no longer directly adjacent to its current position.

Once the full exploration procedure was functional, the remaining work focused on fine-tuning. One observed issue was stuttering caused by the controller, which introduced noise into the distance measurement and therefore affected the reliability of cell-centre detection. A misestimation of the cell centre occurred particularly during longer driving periods, as the accumulated error increased over time. This required further adjustment of the controller behaviour and the related thresholds. After that, the robot speed was increased and the previous stop-and-check strategy was removed, so that planning and decision making could take place while the robot was still driving. This represented a more realistic operating mode and better matched the intended final behaviour of the system.

Finally, different maze layouts were tested in order to verify that the robot was able not only to explore reliably, but also to use the generated map correctly for the final run. In particular, these tests were used to check whether the robot consistently selected the optimal route to the predefined goal after exploration had finished.

VII. Results

The results of the project are best understood as the final achieved behaviour of the implemented Micromouse system after the testing and tuning process described in the previous chapter. For this reason, the present section does not repeat the validation procedures in detail, but instead summarizes the final technical outcomes of the sensing, control, and navigation development. Overall, the project resulted in a functioning autonomous robot that was able to combine wall detection, closed-loop motion control, and map-based maze traversal in a single embedded platform.

A. Sensor Results

The infrared sensing system produced reliable wall-detection signals that were suitable for both control and navigation tasks. In practical operation, the most important outcome was not the exact reconstruction of metric wall distance, but the dependable distinction between the presence and absence of nearby maze walls. For this purpose, a sensor value of 800 proved suitable as the final wall-detection threshold. This made it possible to use the side sensors for wall-based trajectory correction and the front sensor for front-wall detection and cell interpretation.

The sensor measurements also showed that the practically relevant operating range inside the maze was smaller than the full nominal range suggested by the datasheet. In free measurements, raw values reached approximately 3300, whereas in realistic maze conditions the side sensors reached only about 2300 at very close wall distances. Nevertheless, the available signal range was fully sufficient for the implemented purpose. As a result, the sensing subsystem can be regarded as successful, since it provided stable and usable input signals without requiring major redesign.

B. Controller Results

The drive controller achieved stable closed-loop wheel-speed regulation and sufficiently accurate straight-line motion for the intended maze task. After tuning, the final wheel-speed controller operated with the parameter values $K_P = 1.4$ and $K_I = 16.0$ at a sampling time of $T_s = 0.01\,\mathrm{s}$. The same PI parameter set was used for both wheels, since this provided sufficiently similar behaviour on the left and right side while keeping the implementation simple. As summarized in Appendix A, lower proportional gains produced little visible regulating effect, whereas substantially higher gains led to increasingly irregular and stuttering behaviour; the selected final parameters therefore represented the most useful compromise observed during empirical tuning. With these settings, the motors tracked commanded speeds reliably enough for repeated straight driving and for the execution of the higher-level navigation functions.

In addition to the inner wheel-speed loop, the wall-based outer correction layer produced a clear practical improvement in corridor tracking. The final implementation used a proportional trim controller with gain $K_P = 0.004$, because this gave better behaviour than the initially considered PI-based trim correction. In practice, the proportional correction avoided the pronounced oscillatory motion observed previously and provided a more direct and stable correction of lateral deviations. The final implementation applied wall-based correction only when a valid side-wall reference was available, used a target side-sensor value of 1130, and employed low-pass filtering with $\alpha = 0.50$ for the measured wheel speed and $\alpha = 0.30$ for the trim signal. In addition, the drive command was slew-rate limited to a maximum change of 0.07 per control step. Together, these measures led to sufficiently smooth drive behaviour for maze operation. The final default drive speed of the implemented system was $500\,\mathrm{mm/s}$.

C. Navigation Results

The final navigation software was able to combine local sensor-based decisions with a persistent internal maze representation. In operation, the robot could detect cell transitions,

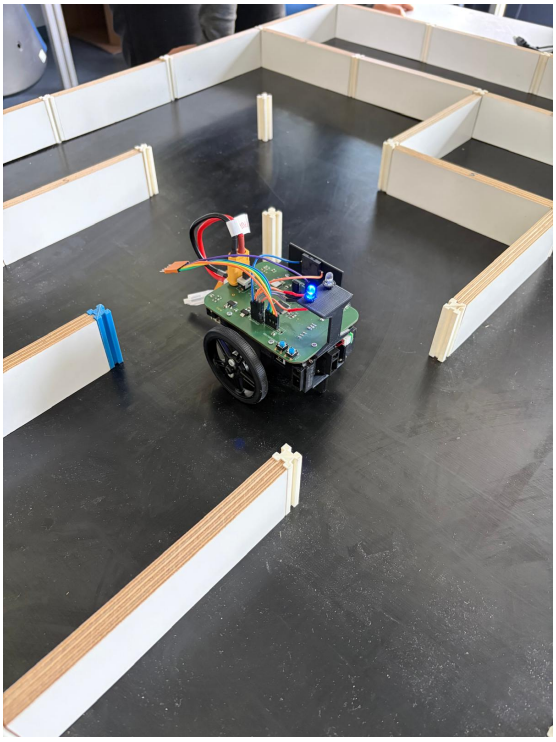

Fig. 7. Micromouse operating inside the reduced test maze.

store neighbouring cell connections, and maintain a consistent orientation estimate sufficiently well for incremental exploration of the reduced 6×6 maze. This confirmed that the combination of encoder-based distance estimation and front-sensor-based cell interpretation was adequate for the reduced project scope.

After the exploration phase, the robot was able to use the generated map for purposeful movement through already explored maze regions. In particular, the implementation was capable of returning after exploration and, on the following run, using the stored map to calculate and follow a path toward the predefined goal. The navigation result is therefore significant not because the system achieved maximum speed, but because it demonstrated the complete functional chain from sensing and control to localisation, mapping, and goal-directed motion.

Figure 7 shows the Micromouse inside the reduced test maze during operation. This image is worth including because it documents the final integrated system in its actual environment and gives the reader a direct impression of the physical scale and testing context of the achieved navigation behaviour.

D. Overall System Result

Taken as a whole, the project achieved its primary goal of producing a functioning Micromouse prototype that integrates electronics, embedded software, control, and navigation in one system. The robot was able to detect maze walls, regulate its wheel motion in closed loop, explore a reduced maze representation, and reuse the generated map for a later goal-directed run. This confirms that the implemented concept was technically viable and that the selected architecture was appropriate for the intended task.

At the same time, the results also show the limits of the current implementation. The realised platform should be understood as a functional academic prototype rather than as a fully competition-optimised high-speed Micromouse. Nevertheless, within the intended scope of the project, the achieved behaviour demonstrates that the essential design objectives were met and that the robot operated successfully as an autonomous embedded navigation system.

VIII. Problems and Discussion

During the development of the Micromouse robot, several practical problems were encountered that affected commissioning, debugging, and system integration. Some of these issues were resolved completely, while others remained as limitations of the final system behaviour. For this reason, the discussion in this chapter addresses not only the most relevant implementation problems, but also the main technical limits of the realised prototype. This distinction is important because the project should be evaluated not only by whether the robot functioned at all, but also by how robustly and under what constraints it was able to perform the intended task.

A. Turning Accuracy and Motion Robustness

One important limitation of the final system concerns turning behaviour. Straight driving was supported by closed-loop wheel-speed control and wall-based trim correction, but the turning manoeuvres themselves were executed with fixed motor commands and were terminated once a target encoder count had been reached. This approach was sufficient to realise left, right, and 180° turns in practice, but it did not constitute full closed-loop orientation control during the turn itself.

As a consequence, the turning accuracy depended on factors such as floor friction, battery condition, wheel slip, and the exact mechanical response of the drivetrain. In practice, the turns were therefore only approximate. The robot was still able to recover from moderate errors once it re-entered a corridor with side-wall reference, because the wall-based correction could compensate some of the resulting misalignment. However, this also means that the straight-line controller partly compensated for imperfections that had actually originated in the turning process. From an engineering perspective, this is acceptable for a functional academic prototype, but it is also a clear limit of the present implementation. A more advanced system would ideally use dedicated closed-loop turning control, for example with more precise turn calibration or an additional orientation reference.

B. Cell-Centre Detection and Localisation Sensitivity

A second important limitation concerns the detection of cell centres and the resulting localisation consistency. The navigation software estimates the next cell centre using a combination of encoder-based travelled distance and front-sensor-based wall interpretation. This concept is technically reasonable because it combines internal motion estimation

with external environmental information. Nevertheless, the method remains sensitive to disturbances in the underlying signals.

In practice, the reliability of cell-centre detection was influenced by controller-induced speed fluctuations, accumulated odometry error, and the threshold-based interpretation of the front sensor. If the decision was triggered slightly too early or too late, the internal cell index of the robot no longer matched its physical position exactly. Since the maze map is built incrementally from these position updates, even a small localisation error can affect the consistency of the later navigation decisions. This was one of the main reasons why the final system should be understood as a robust educational prototype rather than as a fully optimised competition platform. The implemented approach worked sufficiently well for the reduced maze and moderate operating speed, but its sensitivity would become more critical in longer runs or at higher speed.

C. Reduced Maze Scope and Algorithmic Generality

Another important point for interpreting the project results is the deliberately reduced problem scope. The classical Micromouse task is defined on a 16×16 maze, whereas the present project used a 6×6 maze with a predefined goal. This reduction was justified in the context of the available time and resources, because it made it possible to complete the full chain of hardware design, embedded implementation, control design, testing, and autonomous navigation within one academic project.

At the same time, this decision limits the generality of the final results. The implemented navigation software demonstrated successful map construction and goal-directed path execution, but it was not validated on the full classical problem size and should therefore not be interpreted as a competition-level Micromouse solution. In particular, larger mazes place higher demands on localisation robustness, systematic exploration efficiency, and overall repeatability of motion. The reduced scope was therefore an appropriate engineering trade-off, but it remains important to state explicitly that the reported results refer to this reduced task definition.

Besides these system-level limitations, several concrete implementation problems also occurred during commissioning and hardware integration. These are discussed in the following subsections because they significantly influenced the practical development effort and the available debugging workflow.

D. ICSP Programming Interface Error

A first major problem occurred during the initial programming of the microcontroller. Although the ICSP interface had been included in the schematic and PCB design, the component used in Eagle did not match the exact programming device available during implementation. As a result, the pin assignment on the board did not correspond correctly to the required ICSP signals. In particular, the supply pins and the programming-related connections were assigned incorrectly, so that the expected programming signals were not applied to the intended contacts. Consequently, the microcontroller could not be flashed successfully during the first commissioning attempts.

To diagnose and circumvent this problem, the fixed pin-header arrangement at the programming interface was temporarily replaced by individually routed jumper wires. This made it possible to rearrange the signal assignment manually between the board and the programmer until a working connection was found. In this way, the incorrect PCB-level mapping could be compensated externally without immediately redesigning the board. The workaround ultimately enabled successful flashing and therefore allowed further software development to continue.

A second complication arose during the rework of this interface. When the original header arrangement was removed, excessive heat damaged two solder pads, including the originally intended `VDD` connection. As a result, this contact could no longer be used reliably. The problem was resolved pragmatically by sourcing the required supply connection from another header on the same board that belonged to a different module of the system, while the remaining programming-related lines continued to use the ICSP area. Although this solution was improvised, it restored a functioning programming interface and remained usable for the remainder of the project.

Overall, this problem highlighted the importance of verifying not only the logical schematic design, but also the exact correspondence between library components, physical connectors, and the programming hardware used during commissioning. In addition, the incident showed that rework on densely soldered interfaces can easily introduce secondary damage if not carried out carefully.

E. Bluetooth Module Configuration Error

A further hardware-related problem occurred during the integration of the Bluetooth module. In this case, the issue was caused by confusion between the pins `P2_0` and `P0_2`, whose names are visually similar but whose functions differ substantially. One of these pins was intended to define the operating mode of the module and therefore had to be held permanently at a defined logic level, while the other was associated with a status indication function [4]. During implementation, these two pins were confused, and the LED-based indicator circuit was connected to the wrong signal.

As a consequence, the mode-selection pin did not receive a stable 3.3 V high level as required for normal application operation. Instead, the connected LED path interfered with the intended signal level, so that the Bluetooth module did not operate correctly. The error was eventually identified by revisiting the module pin assignment and comparing the implemented connection against the intended configuration.

The practical fix was straightforward but irreversible in terms of debugging convenience. The resistor and LED associated with the incorrect connection were removed, and the line was bridged directly with solder in order to establish a permanent electrical connection. This modification restored the required logic level and made the Bluetooth module operational. However, the original LED-based activity indication was no longer available afterward. The problem therefore did

not prevent further use of the communication interface, but it did reduce the available visual debugging support.

This issue illustrates how easily implementation errors can arise when documentation contains similarly named pins with very different roles. In such cases, even a small interpretation error in the pinout can render an otherwise correctly assembled module inoperative.

F. BLE Communication and Range Limitations

Another recurring difficulty concerned the practical use of the Bluetooth-based UART interface during debugging. Even after the hardware connection of the module had been corrected, it was initially unclear how the communication between the BLE-capable robot hardware and an external device should be accessed and interpreted in practice. In particular, the expected data stream was not immediately available through a simple terminal-like workflow on a laptop, and the interaction between the BLE peripheral on the robot and the corresponding host-side interface required additional investigation.

Two practical solutions were identified during development. One option was to use a BLE scanner application, specifically *nRF Connect for Mobile*, which provides a general-purpose interface for discovering and inspecting Bluetooth Low Energy devices [11]. This allowed the transmitted data to be observed directly on a mobile device. The second solution was the development of a custom Python-based host script on the laptop side, which was ultimately adopted for the intended debugging workflow. This script provided a more convenient and reproducible way of accessing the transmitted data in the development environment.

In addition to the software-side communication difficulty, a practical limitation of the wireless interface was observed in the physical range of the Bluetooth connection. In operation, the connection proved to be reliable only over a relatively short distance of approximately 0.5 m. This was significantly less than initially desired for comfortable debugging and monitoring. It was suspected that the PCB environment, in particular the surrounding conductive structures and ground-plane configuration, negatively affected the radio performance of the module [4].

Taken together, these communication-related issues did not prevent the use of the wireless debugging interface, but they did increase the implementation effort and reduced the practical convenience of the solution. The experience showed that integrating BLE communication is not only a matter of electrical connectivity, but also of understanding the host-side software workflow and the physical limitations imposed by the final hardware environment.

IX. Conclusion

The aim of this project was to design, implement, and evaluate a functioning Micromouse robot capable of autonomous maze navigation within a reduced academic project scope. Within this scope, the aim was achieved. The final system combined a custom embedded hardware platform with closed-loop motion control, infrared wall sensing, encoder-based odometry, and map-based navigation software, and it was able to explore a reduced 6×6 maze and use the generated map for a later goal-directed run. As a result, the project demonstrated that the essential tasks of sensing, control, localisation, and navigation could be integrated successfully into a single autonomous robotic platform.

From a technical point of view, the main achievement of the project was not maximum speed, but the successful integration of multiple engineering disciplines into one coherent system. The implemented hardware architecture, based on a dsPIC-centered PCB with dedicated sensing, drive, communication, and power subsystems, proved suitable for this purpose. In addition, the layered software structure and the cascaded controller design provided an effective basis for organising the real-time behaviour of the robot and for managing the interaction between low-level actuation and high-level navigation.

At the same time, the project also revealed the limits of the current implementation. The robot should be understood as a functional academic prototype rather than as a fully optimised competition Micromouse. The most important technical limitations were the approximate open-loop turning behaviour, the sensitivity of cell-centre detection and localisation to motion disturbances, and the deliberately reduced maze scope. Several practical issues during development, including the programming-interface error, Bluetooth integration problems, and limited wireless range, also increased the implementation effort and reduced debugging convenience. Taken together, these points show that the implemented concept was technically viable, but that further refinement would be required for a more demanding and general Micromouse system.

A. Outlook

Several directions for future work follow directly from the present results. A first important step would be the extension of the navigation software from the current reduced maze representation to the full classical 16×16 Micromouse problem. In parallel, the motion performance could be improved through dedicated closed-loop turning control, more robust localisation, and continued refinement of the wall-based trajectory correction. On the hardware side, a revised programming interface and improved wireless communication layout would increase maintainability and debugging convenience. Taken together, these improvements would build naturally on the implemented prototype and could transform it from a successful educational platform into a more capable and competition-oriented autonomous robot.

REFERENCES

[1] Dr. Fritz Faulhaber GmbH & Co. KG, *DC-Getriebemotoren, Serie 2619...SR...IE2-16: Edelmetallkommutierung mit integriertem Encoder*, Faulhaber, 2018, datasheet, Edition 2018.

[2] Sharp Corporation, *GP2Y0A51SK0F Distance Measuring Sensor Unit: Analog Output Type, Measuring Distance 2 to 15 cm*, Sharp, datasheet, Sheet No. OP13007EN.

[3] Microchip Technology Inc., *dsPIC33FJXXXMCX06A/X08A/X10A Motor Control Family Data Sheet*, https://www.mouser.de/datasheet/3/282/1/70291G.pdf, Microchip Technology Inc., datasheet, document 70291G.

[4] ——, *RN4870/71 Bluetooth® Low Energy Module*, Microchip Technology Inc., 2023, data sheet, document DS50002489H.

[5] Texas Instruments, *LM340, LM340A and LM7805 Family Wide VIN 1.5-A Fixed Voltage Regulators*, https://cdn.sparkfun.com/assets/1/7/7/3/2/LM7805.pdf, Texas Instruments, datasheet.

[6] STMicroelectronics, *LD1117A Adjustable and Fixed Low Drop Positive Voltage Regulator*, https://www.st.com/resource/en/datasheet/ld1117a.pdf, STMicroelectronics, datasheet.

[7] Toshiba Electronic Devices & Storage Corporation, *TB6612FNG Toshiba Bi-CMOS Integrated Circuit Silicon Monolithic*, https://cdn.sparkfun.com/datasheets/Robotics/TB6612FNG.pdf, Toshiba Electronic Devices & Storage Corporation, datasheet.

[8] G. Müller and B. Ponick, *Grundlagen elektrischer Maschinen*, 10th ed., ser. Elektrische Maschinen. Weinheim: Wiley-VCH, 2014.

[9] D. Schröder, *Elektrische Antriebe*, 4th ed., ser. Elektrische Antriebe. Berlin u.a.: Springer, 2015.

[10] S. V. Vaseghi, *Advanced Signal Processing and Digital Noise Reduction*. Wiesbaden: Vieweg+Teubner Verlag, 1996.

[11] Nordic Semiconductor ASA, "nRF Connect for Mobile," https://www.nordicsemi.com/Products/Development-tools/nRF-Connect-for-mobile, 2026, mobile application for scanning, exploring, and communicating with Bluetooth Low Energy devices. Accessed: 2026-04-18.

APPENDIX A
PI TUNING SUMMARY

Empirical tuning sequence for the inner PI wheel-speed controller. The table summarizes the qualitative observations obtained during incremental gain adjustment.

Wheel	K_P	K_I	Test Sequence [mm/s]	Observed Behavior	Assessment
Coarse proportional tuning (right wheel, free-spinning)					
Right	0.0	0.0	0 / 500	No visible control action; the motor response followed the command without meaningful regulation.	Unsuitable baseline
Right	0.2	0.0	0 / 500	No relevant improvement over the baseline case; the controller effect remained negligible.	K_P too low
Right	0.5	0.0	0 / 500	Still no effective regulation; the commanded speed was not tracked sufficiently.	K_P too low
Right	1.0	0.0	0 / 500	First visible regulating effect; however, the measured speed still remained noticeably offset from the target.	Improved, but insufficient
Right	1.3	0.0	0 / 500	Regulation became more apparent and more stable, but the target velocity was still not reached accurately enough.	Close, but still off target
Right	1.4	0.0	0 / 500	Stable proportional response with acceptable speed tracking and no pronounced oscillation.	Selected proportional gain
Right	1.5	0.0	0 / 500	Clearly visible stuttering appeared during operation, indicating the onset of unstable behavior.	K_P too high
Right	2.0	0.0	0 / 500	Strong stuttering and highly irregular drive behavior.	Rejected
Integral tuning after proportional gain selection (right wheel, free-spinning)					
Right	1.4	0.0	0 / 500 / 700 / 300 / 0	Stable proportional control, but remaining steady-state deviation was still visible at different command levels.	Basis for integral tuning
Right	1.4	1.0	0 / 500 / 700 / 300 / 0	No significant change compared with the P-only case.	K_I too low
Right	1.4	2.0	0 / 500 / 700 / 300 / 0	Only minor improvement; the effect of the integral term remained limited.	K_I too low
Right	1.4	5.0	0 / 500 / 700 / 300 / 0	Slightly improved correction of offset, but no major change in overall behavior.	Still below optimum
Right	1.4	10.0	0 / 500 / 700 / 300 / 0	Stable response across the speed sequence, though the additional benefit remained moderate.	Acceptable, but not optimal
Right	1.4	15.0	0 / 500 / 700 / 300 / 0	Noticeable improvement in target tracking while maintaining stable behavior.	Promising range
Right	1.4	16.0	0 / 500 / 700 / 300 / 0	Good compromise between tracking accuracy and smooth response over the full command sequence.	Final parameter set
Right	1.4	17.0	0 / 500 / 700 / 300 / 0	Stuttering began to reappear, indicating that the integral action had become too aggressive.	Slightly too high
Right	1.4	20.0	0 / 500 / 700 / 300 / 0	Strong stuttering and visibly unstable drive behavior.	Rejected
Transferred tuning and confirmation on left wheel					
Left	1.0	10.0	0 / 500 / 700 / 300 / 0	Initial transferred starting point based on the tuning experience from the right wheel; regulation was already functional but not yet optimal.	Reasonable starting estimate
Left	1.2	10.0	0 / 500 / 700 / 300 / 0	Stable response, but tracking remained somewhat less precise than desired.	Improved, but still conservative
Left	1.4	10.0	0 / 500 / 700 / 300 / 0	Good proportional response with similar behavior to the right wheel before final integral tuning.	Suitable proportional region
Left	1.4	15.0	0 / 500 / 700 / 300 / 0	Noticeable improvement in target tracking while retaining stable behavior.	Near final range
Left	1.4	16.0	0 / 500 / 700 / 300 / 0	Left wheel showed similarly acceptable behavior and converged quickly to the same usable parameter set as the right wheel.	Confirmed common gains
Left	1.4	17.0	0 / 500 / 700 / 300 / 0	Slight onset of stuttering comparable to the right-wheel observations.	Slightly too high

APPENDIX B
PCB SCHEMATIC

The complete circuit schematic of the Micromouse electronics is included here as supporting hardware documentation. It complements the hardware design chapter by showing the detailed electrical connections between the controller, power supply, sensors, motor driver, communication interface, and auxiliary components.

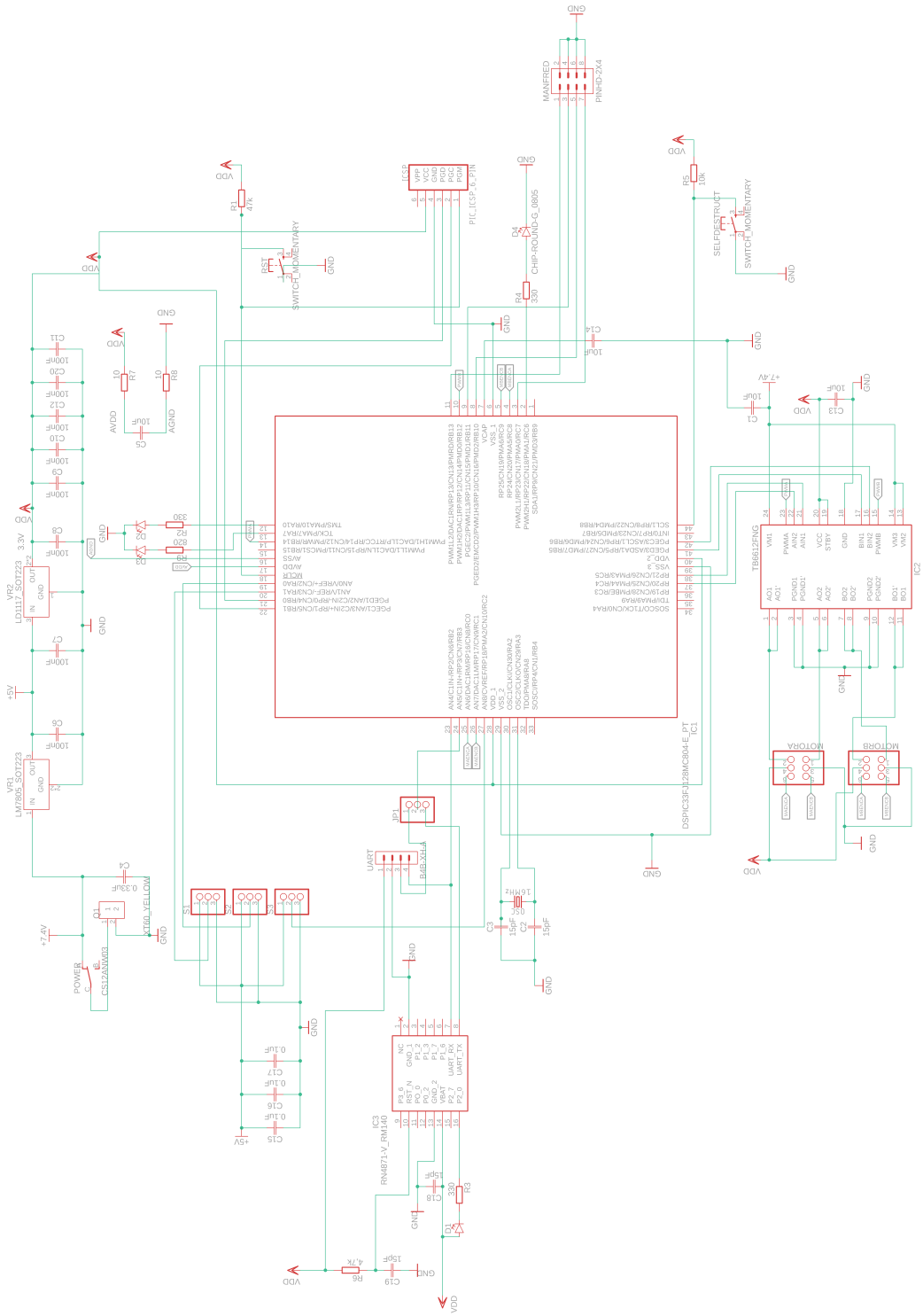

Figure 8. Schematic of the Micromouse electronics.

APPENDIX C
BILL OF MATERIALS

The bill of materials is summarized here in a single rotated table for improved readability. This keeps the component summary and procurement information together while giving the long manufacturer and part-number fields more horizontal space.

Table I. Summary of the bill of materials for the Micromouse PCB.

Reference	Qty	Description	Value	Manufacturer	Part Number
PCB	1	PCB board	Board	—	—
C1, C5, C13, C14	4	Capacitor - Generic	10uF	TDK	CGA4J1X7R0J106K125AC
C2, C3, C18, C19	4	Capacitor - Generic	15pF	KEMET	C0805C150J5HACTU
C4	1	Capacitor - Generic	0.33uF	—	—
C6, C7, C8, C9, C10, C11, C12, C20	8	Capacitor - Generic	100nF	KEMET	C0805C101J5GAC7800
C15, C16, C17	3	Capacitor - Generic	0.1uF	KEMET	C0805C101J5GAC7800
D1, D4	2	Green LED - Generic	CHIP-ROUND-G_0805	Kingbright	APT2012LZGCK
D2	1	Blue LED - Generic	CHIP-ROUND-B_0805	Kingbright	APT2012LVBC-D
D3	1	Red LED - Generic	CHIP-ROUND-R_0805	Kingbright	APT2012LSECK-J3-PRV
IC1	1	16-bit dsPIC microcontroller	DSPIC33FJ128MC804-E_PT	Microchip Technology	dsPIC33FJ128MC804-E/PT
IC2	1	Dual H-bridge motor driver	TB6612FNG	Toshiba	TB6612FNGC8EL
IC3	1	Bluetooth Low Energy module	RN4871-V_RM140	Microchip Technology	RN4871-V/RM140
ICSP	1	ICSP programming connector	PIC_ICSP_6_PIN	Molex	22-23-2061
JP1, S1, S2, S3	4	Pin header	1X03	—	—
MANFRED	1	Pin header	PINHD-2X4	—	—
MOTORA, MOTORB	2	Pin header	2X03	—	—
OSC	1	16MHz crystal	16MHz	Abracon	ABL-16.000MHz-B4Y
POWER	1	Power switch	CS12ANW03	NKK Switches	CS12ANW03
Q1	1	XT60 power connector	XT60_YELLOW	AMASS	XT60PB-M
R1	1	Resistor	47k	Vishay Beyschlag	MCU08050C4702FP500
R2, R3, R4	3	Resistor	330	Vishay	TNPW0805330RBYEA
R5	1	Resistor	10k	Bourns	CRT0805-FZ-1002ELF
R6	1	Resistor	4.7k	Vishay Beyschlag	MCU08050C4701FP500
R7, R8	2	Resistor	10	TE Connectivity	CPF0805B10RE
R9	1	Resistor	820	Panasonic	ERA-6VEB8200V
RST, SELFDESTRUCT	2	Tactile switch	SWITCH_MOMENTARY	C&K	PTS647SN70SMTR2N-LFS
UART	1	XH connector top entry - 4POS	B4B-XH-A	JST	B4B-XH-A(LF)(SN)
VR1	1	5V voltage regulator	LM7805_SOT223	Texas Instruments	LM7805MP/NOPB
VR2	1	3.3V voltage regulator	LD1117_SOT223	STMicroelectronics	LD1117AS33TR

Controller	Side	#	PIN Description	5V	I/O (X = undef)	Usage (Name)
dspic33fj128mc804	A (right)	1	SDA1 / RP9 / CN21 / PMD3 / RB9		X	
		2	PWM2H1 / RP22 / CN18 / PMA1 / RC6		O	LED_GREEN
		3	PWM2L1 / RP23 / CN17 / PMA0 / RC7		X	EXTRA7
		4	RP24 / CN20 / PMA5 / RC8		I	ENC_A_RIGHT
		5	RP25 / CN19 / PMA6 / RC9		I	ENC_B_RIGHT
		6	VSS		X	
		7	VCAP		X	
		8	PGED2 / PWM1H3 / RP10 / CN16 / PMD2 / RB10		X	EXTRA5
		9	PGEC2 / PWM1L3 / RP11 / CN15 / PMD1 / RB11		X	EXTRA3
		10	PWM1H2 / DAC1RP / RP12 / CN14 / PMD0 / RB12		O	PWM_RIGHT
		11	PWM1L2 / DAC1RN / RP13 / CN13 / PMRD / RB13		X	EXTRA1
		12	TMS / PMA10 / RA10		O	LED_BLUE
		13	TCK / PMA7 / RA7		O	LED_RED
		14	PWM1H1 / DAC1LP / RTCC / RP14 / CN12 / PMW		O	PWM_LEFT
		15	PWM1L1 / DAC1LN / RP15 / CN11 / PMCS1 / RB15		X	
	B (top)	16	AVSS		X	
		17	AVDD		X	
		18	MCLR		X	
		19	AN0 / VREF+ / CN2 / RA0		I	SENSOR_RIGHT
		20	AN1 / VREF- / CN3 / RA1		I	SENSOR_MIDDLE
		21	PGED1 / AN2 / C2IN- / RP0 / CN4 / RB0		X	
		22	PGEC1 / AN3 / C2IN+ / RP1 / CN5 / RB1		X	
		23	AN4/ C1IN- / RP2 / CN6 / RB2		O	UART_TX
		24	AN5 / C1IN+ / RP3 / CN7 / RB3		I	UART_RX
		25	AN6 / DAC1RM / RP16 / CN8 / RC0		I	ENC_A_LEFT
		26	AN7 / DAC1LM / RP17 / CN9 / RC1		I	ENC_B_LEFT
		27	AN8 / CVREF / RP18 / PMA2 / CN10 / RC2		I	SENSOR_LEFT
	C (left)	28	VDD		X	
		29	VSS		X	
		30	OSC1 / CLKI / CN30 / RA2		X	
		31	OSC2 / CLKO / CN29 / RA3		X	
		32	TDO / PMA8 / RA8		X	
		33	SOSCI / RP4 / CN1 / RB4		X	
		34	SOSCO / T1CK / CN0 / RA4		X	
		35	TDI / PMA9 / RA9		X	
		36	RP19 / CN28 / PMBE / RC3		X	
		37	RP20 / CN25 / PMA4 / RC4		O	MOTOR_LEFT_IN1
		38	RP21 / CN26 / PMA3 / RC5		O	MOTOR_LEFT_IN2
	D (bottom)	39	VSS		X	
		40	VDD		X	
		41	PGED3 / ASDA1 / RP5 / CN27 / PMD7 / RB5		O	MOTOR_RIGHT_IN1
		42	PGEC3 / ASCL1 / RP6 / CN24 / PMD6 / RB6		O	MOTOR_RIGHT_IN2
		43	INT0 / RP7 / CN23 / PMD5 / RB7		X	
		44	SCL1 / RP8 / CN22 / PMD4 / RB8		I	SELF_DESTRUCT

TABLE II

TABLE 2: MICROCONTROLLER PIN LAYOUT. ANALOGUE PINS ARE MARKED BLUE, PWM PINS GREEN, RP PINS ORANGE.